

Relatório — Determinismo e Redução de Custo de Token no Pipeline

Status: implementado, testado e validado com rodada real “depois” na branch `feature/determinismo-pipeline-custos` **Data:** 2026-08-13 **Plano original:** `melhorias/plano-determinismo-reducao-custos.md` **Comparativo visual:** `comparativo-custo-antes-depois.html`

1. O que foi pedido

Mapear campo a campo o que no pipeline (`analista-insumos` → `diretor-de-arte` → `redator-*` → `revisor-marca`) é decisão estrutural fixa (pode virar script) vs. criação de conteúdo real (tem que ficar em LLM); montar um plano; implementar; testar; e medir com uma rodada real antes e uma depois se a mudança realmente reduz custo — não estimado, medido.

2. O que foi implementado

Dois commits de melhoria, cada um só avançou com testes verdes antes do próximo:

1. **Decomposição fixa de objetivo_tom** (`scripts/parametros_projeto.py`) — tabela de 3 pares movida de “escrita por extenso no prompt da skill” para uma função testável (`decompor_objetivo_tom`), com CLI (`--decompor-objetivo-tom`). `diretor-de-arte/SKILL.md` atualizado pra chamar o script em vez de re-derivar.
2. **Pré-filtro determinístico de claims** (`scripts/extrair-claims-candidatos.py`) — extrai números/%/hashtags/datas/nomes-próprios via regex dos `.txt` do material, compara contra `dossie_insumos.md`, grava `output/<slug>/revisao/candidatos_verificacao.json`. `revisor-marca/SKILL.md` atualizado pra rodar isso antes da checagem manual de fidelidade à fonte.

3. Como foi validado (evidência, não opinião)

- **17 testes automatizados** novos (`tests/`, convenção que o `pytest.ini` já previa mas nunca tinha sido populada), incluindo:
 - regressão exata do defeito real encontrado (hashtag de outro nicho colada por engano — a mesma classe de erro que o `revisor-marca` corrigiu numa rodada anterior);
 - regressão de um bug encontrado durante a própria validação (número colado dentro de uma hashtag gerando sub-match espúrio — só apareceu ao testar contra conteúdo real, não nos casos sintéticos iniciais);
 - teste de CLI via subprocess real, não só chamada de função.
- **Guardas determinísticos do próprio repo** (`verificar-universalidade.py`, `verificar-consistencia-pipeline.py`) rodados e verdes depois de cada edição de `SKILL.md`.
- **Validação contra dado real:** o novo script rodado contra `output/zz-teste-painel-view/textos/` (produção real de uma rodada anterior) antes de qualquer alteração no `revisor-marca` — confirmou que conteúdo já correto não gera falso positivo relevante (3 candidatos, todos hashtags combinando termos reais do dossiê de forma legítima, não fabricação).

- **Rodada real “depois”** (/produzir-comunicacao-completa via claude -p, mesmo insumo/material do baseline, slug novo) — concluiu com auditoria CONFORME, pacote de distribuição gerado, nenhum claim fabricado.

4. Resultado medido

Etapa	Antes	Depois	Δ %
Orquestrador (insumos+briefing+dispatch)	175.637	143.899	-18,1%
Revisor de marca	141.131	87.502	-38,0%
Produtor de textos	97.121	94.957	-2,2%
Total pago	413.889	326.358	-21,2%

Tokens extraídos direto dos .jsonl de transcript de sessão real (input + output + cache_creation; exclui cache_read, cobrado a preço reduzido — que também caiu 33,4%). A etapa mais cara do baseline (revisor de marca) foi a que mais encolheu, confirmando a hipótese do plano: o maior retorno estava em pré-filtrar antes do julgamento de LLM, não em tentar determinizar a escrita criativa (produtor de textos, que não foi tocado, ficou praticamente estável — como esperado).

5. Limitações e próximos passos

- O pré-filtro de claims cobre hoje só .txt (material textos). Não cobre copies.json de arte/kits, HTML (landing/apresentação) nem PDF — extensão natural se a economia por material se confirmar em mais rodadas.
- Uma única rodada “antes” e uma “depois” — variância entre execuções de LLM existe (o produtor de textos, por exemplo, teve mais turnos na rodada “depois” mas menos tokens líquidos). Para decisão de produção, valeria repetir mais vezes e olhar a média, não um único ponto.
- O pré-filtro é assistivo por design — nunca vira gate automático (exit 0 sempre), então a economia depende de revisor-marca de fato usar a lista, não ignorá-la.

6. Reversibilidade

Tudo em feature/determinismo-pipeline-custos, sem alterar main. Não fazer merge descarta tudo sem custo.